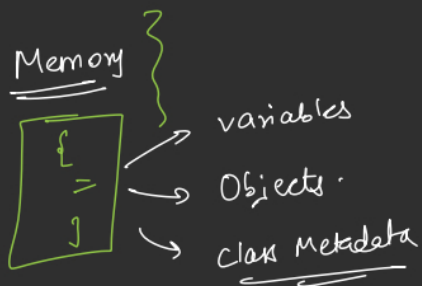


Memory Management



JVM

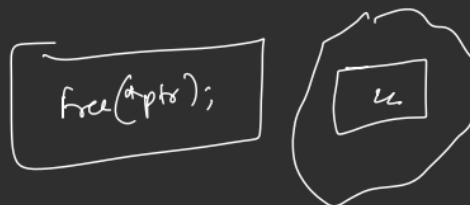
- ① Memory allocate.

int x = 4;

Student s1 = new Student();
- ② Memory use / update.

x = x + 1;
- ③ Memory de-allocate / Free

Manual Memory Management (C/C++)

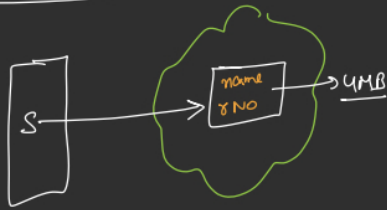


→ Error prone.

Garbage Collection

Why Memory Management?

→ Memory



→ Memory leakage

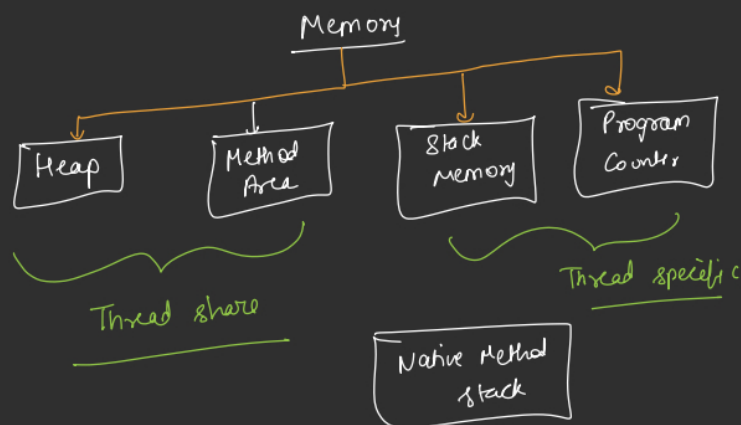
`list<Student> list = new ArrayList<>();`

→ performance get degraded

→ OutOfMemoryError / StackOverflowError
Heap / Stack

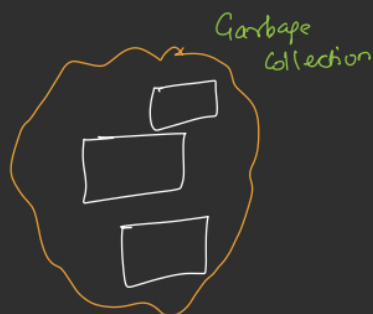
JVM Memory

- Objects
- Threads
- Variables / object → long lived
- Temporary



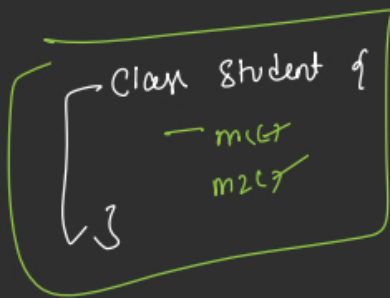
Heap Memory

new



Method Area

↳ class level blue print / Metadata.



— Student

— Object

—

— m1()

— m2()

Static variables

Return type

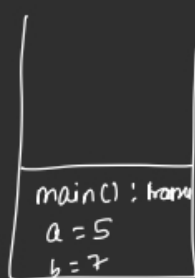
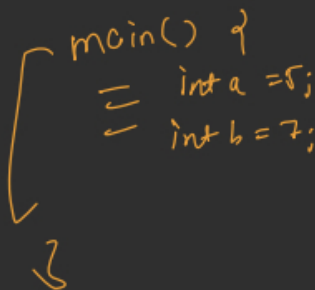
byteCode

↓
(JIT Compile)

Stack Memory

↓ ↳ (LIFO)

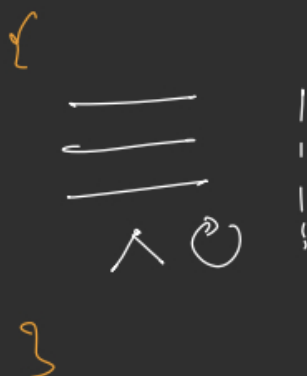
Execution Memory



Stack

Program Counter (PC)

↓
Current instruction



Heap

Stack

Method Area

PC

```

Public class Demo {
    public static void main ( — ) {
        int x = 5;
        Student s1 = new Student();
        s1.name = "Aditya";
    }
}

```

Demo.java — Demo.class

① JVM start.

② Class Loading

Class Loader.

① File Demo.class

② Read the bytecode.

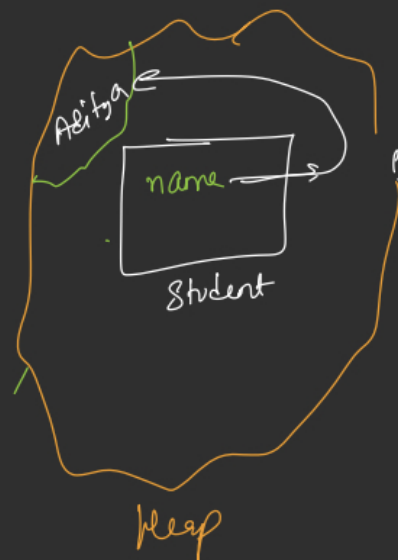
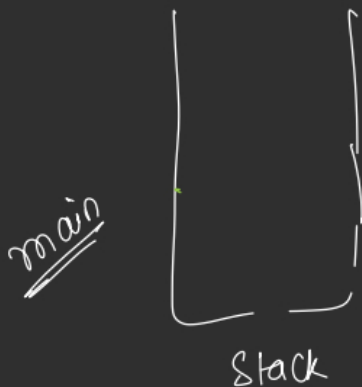
③ Create Method area entry.

③ Main Thread Creates

- ↳ Stack (✓)
- ↳ PC (✓)

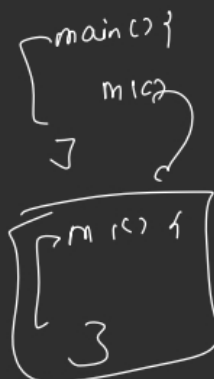
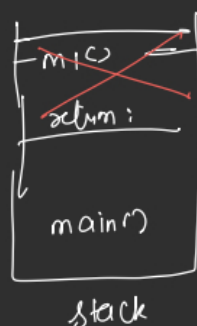
Method Area

Class: Demo
 ↳ parent: Object
 → Method: main (String[])
 bytecode.



Stack Memory in Depth

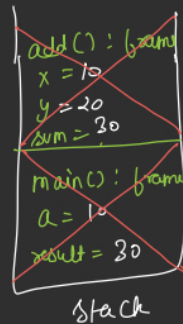
- Local variables.
- Method Params.
- Reference variables.
- Return address



```

Public class Demo {
    void main (String[] args) {
        int a = 10;
        → int result = add(a, 20);
    }
    int add (int x, int y) {
        int sum = x + y;
        return sum;
    }
}

```



Observations

- ↳ Grows & shrink automatically.
- ↳ Memory is auto cleaned.

Stack Overflow Error

- ↳ Common (Recursion w/o base case)

```

F1() {
    F1();
}

```

Heap Memory :

- ↳ data (Objects)
 - milliseconds
 - minutes
 - Program ends.



What Goes in Heap?

→ Objects

Student s1 = new Student();

→ Instance Variable

→ Arrays

→ Strings

class Student {

String name;

int age;



Properties of Heap Memory

→ Shared across Thread.

→ Dynamic in nature

→ -Xms 2m (Minimum space)

→ -Xmx 4m (Max space)

→ Managed by GC.

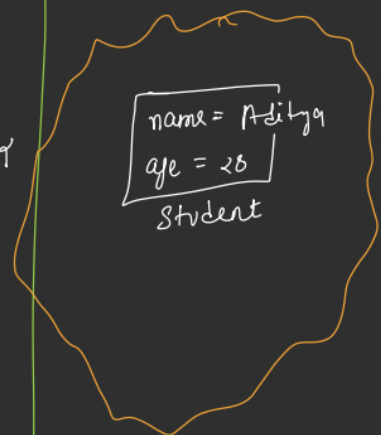
→ Usually slower than stack.

```
class Student {
    String name;
    int age;

    public Student (String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

```
main() {
    Student s1 = new Student
        ("Aditya", 20);
    Student s2 = s1;
}
```

① JVM start

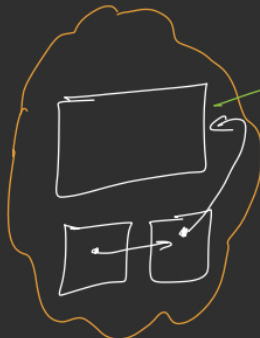


```
class A {  
    B b;  
}
```

```
class B {  
    C c;  
}
```



Stack



Heap

} eligible
for
GC
→

```
{  
    Student s1 = new Student();  
    s1 = null;  
}
```